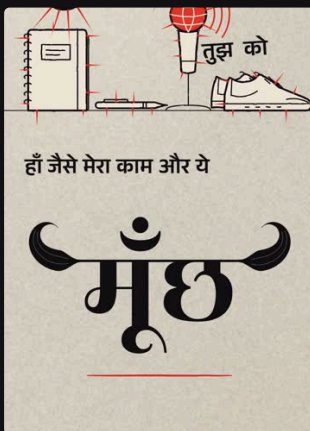


PROCESS GUIDE • CLAUDE CODE × REMOTION

How we made a motion-designed lyric video with AI

प्यार?

The complete workflow, the exact prompts, and the lessons from turning a Hindi rap track and its lyrics into an art-directed 9:16 music video: *PYAAR?* by Naam Sujal (MTV Hustle 4).



Made with Claude Code • October 2026 • Use this guide to make your own.

The idea in one page

We gave Claude Code two files, a song and its lyrics, and directed it through six stages. Each stage ended with something we could **watch or read and approve** before moving on. That gate is the most important part of the process. Each approved stage became the foundation for the next, so the whole build never rested on unchecked work.

STAGE 0 Inspect the inputs Check the song and lyrics files. → file report	STAGE 1 Word-level timing Align your exact lyrics to the vocal, word by word. → word-timestamps.json, phrases.json	STAGE 2 Sync test video A plain black video that highlights each word, to check timing by eye. → sync-test.mp4
STAGE 3 Creative direction Interpret every line; design the visual language and a screenplay. → creative-direction.md, visual-screenplay.json, beat-map.json	STAGE 4 Engine + hero proof Build the animation system; render one short section to set the quality bar. → hero-proof.mp4	STAGE 5 Sections Render longer sections on the same engine, then review. → section-1m20-2m30.mp4

What you need to start

INPUT	NOTES
The song (mp3/wav)	Original mix, full length. Claude separates the vocals itself.
The lyrics (lyrics.txt)	One line per lyric line, in the original script (Devanagari, Latin, mixed — all fine). These are the <i>source of truth for the words</i> .
Claude Code	Running in the project folder (terminal or VS Code). It installs everything else itself.
A Mac/PC with ~5 GB free	For Python models (~2.5 GB) and the Remotion/Chrome renderer.

The three rules that made it work

1 • FREEZE WHAT'S BEEN APPROVED

Once the timing passed review, we told Claude to **freeze** the timestamp files and the audio. Every later stage read them but never changed them. Claude recorded checksums so this could be proven.

2 • REVIEW A PICTURE, NOT CODE

Each stage ends with something you watch: a test video, contact sheets of frames, a short proof render. Claude also inspected rendered frames itself and fixed what looked wrong before showing us.

3 • PROVE QUALITY ON A SMALL PIECE FIRST

We didn't render the whole song straight away. A 5-second "hero proof" set the quality bar, and only after approving it did we scale up to a 70-second section.

01 Word-level timing

≈15 min · fully automatic

Goal: every word of *your* lyrics gets an exact start and end time, measured from the real vocal. This drives everything that follows.

The prompt (full text in the Appendix, prompt B)

```
Before creating any video, I need you to create highly accurate word-level timestamps for the lyrics. ... Prefer
WhisperX or another forced-alignment approach suitable for fast rap vocals.
The lyrics.txt file is the source of truth for the WORDS. The audio is the source of truth for the TIMING.
Do not simply transcribe the song and replace my lyrics. ... Save as word-timestamps.json ... Also create
phrases.json ...
At the end, verify that the timestamps cover the song correctly and report any portions where alignment
confidence appears poor.
```

What Claude actually did

STEP	TOOL	WHY
1. Isolate the voice	Demucs (htdemucs)	The beat confuses speech models; separating vocals made alignment far more accurate.
2. Force-align your words	torchaudio MMS_FA + uroman	A multilingual (1,100+ languages) aligner. Lyrics are romanized, then matched to the vocal. Handles Hindi + English + Marathi in one pass, which WhisperX's Hindi-only model would not.
3. Absorb ad-libs	"Wildcard" token between lines	Unlisted ad-libs and breaths are soaked up instead of dragging nearby words out of place.
4. Independent cross-check	Whisper large-v3 / turbo	A separate transcription confirms timings line by line (agreement within ≈0.3 s).
5. Clean-up & report	Custom script	Extends held/elongated words, trims swallowed echoes, closes tiny gaps, scores every line high/medium/low.

OUTPUTS

word-timestamps.json (601 words, each {word, start, end}), phrases.json (82 lines with their words), and a verification report listing low-confidence spots (sung melodic parts, heavily processed hooks) and audio regions not covered by lyrics.

TIP

Expect some low confidence on **sung/melodic** parts and **effected vocals**. That's normal, and the report flags exactly where to look. The later designs used "forgiving" animation in those spots.

02 Sync test video

Goal: check the timing with your own eyes and ears before anyone designs anything.

A plain black 1080×1920 video: the current line in the middle, the sung word highlighted, and debug text at the bottom (song time, line number, word, start→end). No effects at all.

Key instructions (Appendix, prompt C)

This video's ONLY purpose is synchronization QA.
Use the existing timestamps exactly as they currently exist. Do not silently alter them.
Stop after sync-test.mp4 is rendered.

Watch for: words lighting up early or late, long gaps, and the flagged low-confidence sections.

Gotcha we hit: Hindi needs a text shaper (libraqm/HarfBuzz) or the vowel marks render wrong. Claude installed it and checked a still frame before rendering.



sync-test.mp4: the active word highlighted, with debug info.

03 Creative direction

the most important stage

Goal: turn the lyrics into a *visual language*. The core instruction was: "**The meaning of the lyric should drive the visual.**" Don't zoom one word and slide the next; work out what the line means and how that meaning could exist on screen.

What the prompt asked for (Appendix, prompt D)

- Analyse all 82 lines: meaning, slang, code-switching, cultural context.
- Think **word** → **phrase** → **scene** → **sequence** → **song**: continuous visual storytelling, not 82 separate animations.
- Transitions must have **motivation**: objects from scene A create scene B.
- Analyse the audio for structure, energy, beats and silences; design an **intensity curve** with contrast.
- Deliver `creative-direction.md` + `visual-screenplay.json`, then **self-review**: reject generic, repetitive or template ideas.

What came back

DECISION	RESULT FOR PYAAR?
Concept	"The Case File of a Question." The song argues its case with evidence (pages, headlines, a passbook, judges), and the body keeps bleeding through it.
Three worlds	PAPER (the argument), NIGHT (the feeling, one amber light), BLOOD-RED (reserved for the final "तलब" section).
Signature device	The red line , the writer's pen. It becomes an underline, suture, thread, crack, clock hand, fork in the road and wound.
Typography	5 tested fonts with jobs: Rozha One (headlines), Anek Devanagari variable (kinetic), Tiro (intimate), Kalam (handwriting), Space Mono (data).
Structure	82 lines grouped into 37 sequences ; an intensity curve built around the song's three real silences.
Audio analysis	<code>beat-map.json</code> : 123 BPM, downbeats, kicks, snares, energy curves, section boundaries.

WHY THIS STAGE MATTERS

The best moments came from **meaning and wordplay**, not effects:

- कांटा means both "thorn" and "clock hand", so a thorn-tipped red clock hand freezes on रोक ("stop").
- अदा + लतों = अदालतों: "charms" and "addictions" fuse into "courts".
- The beat really drops out exactly where he sings आँखें बंद कर दे ("close my eyes"), so the frame becomes an eye that closes.

TIP: FONTS

Ask Claude to render a **test sheet** of every candidate font with the hardest words in your lyrics before choosing. It also caught that one font draws the Devanagari numeral ५ like a Latin "5".

04 Engine + hero proof

Remotion · React · TypeScript

Goal: build a reusable motion-design engine, then prove the visual quality on one short, signature section before scaling up.

The engine (folders in `video/src/`)

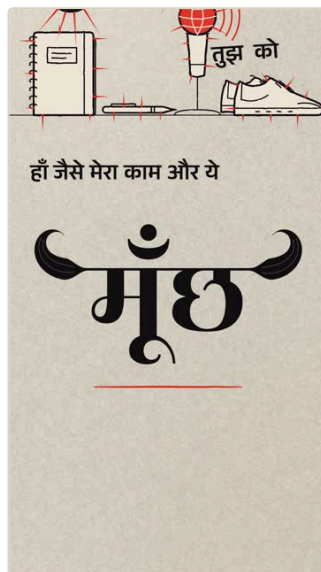
<code>timing/</code>	Reads the frozen JSON; one song clock
<code>typography/</code>	Fonts, akshara-safe text splitting, kinetic lines, ink reveals
<code>graphics/</code>	The red line; original line-art (hand, mic, coin...); moustache
<code>effects/</code>	Spikes, bites, crumble-to-dust, print misregistration
<code>camera/</code>	2.5D camera with parallax layers
<code>backgrounds/</code>	Paper, Night, film grain
<code>sequences/</code>	One file per screenplay scene
<code>utils/</code>	Easing families, physics springs, seeded randomness, geometry

Design rules the prompt enforced

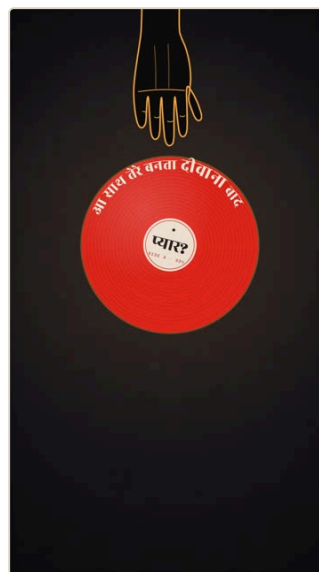
- **Materials move differently.** Paper, ink, thread, rubber, metal and skin each have their own spring physics.
- **No "AI subtitle" look.** No linear slides, no scale-from-zero, not everything centred.
- **Real Devanagari shaping.** Text is split into aksharas, never into raw characters; this was tested on all 601 words.
- **Reels safe zones** for critical text; decoration may bleed.
- **Deterministic renders**, with no random values unless seeded.



"चुभती": needles grow from the real letter outline.



"मुँछ": the word's own top bar curls into a moustache.



Hook: the circle becomes a spinning red record.



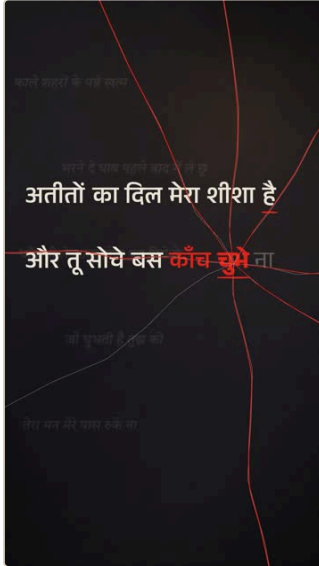
The night was a black card on a desk; it's yanked away.

COURSE CORRECTION WE MADE MID-WAY

The original brief asked for a 42-second hero proof. Partway through, we asked: *"can we make a small section of the video quickly with all this & not take too much time"*. Claude cut the scope to the 5-second moustache moment and rendered it. **Lesson:** a short, polished proof gets you a decision faster than a long, unfinished one.

05 Sections

After approving the proof, one short prompt ("now i want from 1:20 minute to 2:30 minute") produced 15 scenes across 70 seconds on the same engine. Claude reviewed contact sheets of frames at every key moment, fixed issues (wrong arc geometry, text cut off at the frame edge, mirrored text that looked broken), and only then did the full render.



Glass heart cracks on "चुभे".



Chalk outline + verdict stamps.



The red line as a clock hand.



"दो": two fathers, two panels.

Lessons & tips for your own video

DO THIS	BECAUSE
Give exact lyrics and say they're the source of truth	Otherwise you get a transcription that "corrects" your words.
Always make the sync-test video	It's cheap, and timing bugs found later are expensive.
Freeze approved files (and say so in every prompt)	It stops later stages from quietly drifting the timing.
Ask for meaning before motion	"What is the rapper saying, and how could that exist on screen?" gives far better ideas than "add effects".
Ask Claude to self-review its own concepts	It removed repeated stamps, an extra eye motif and duplicate "falling" moves before we ever saw them.
Approve a short proof before scaling	You lock the look on 5 seconds instead of 3 minutes.
Ask for representative frames, not just the video	Stills are easier to critique and share.
For non-Latin scripts, demand shaping tests	Hindi vowel signs and conjuncts break easily; test sheets catch it.
Name the platform (Reels/Shorts)	Critical text then avoids the caption, button and header areas.

Reuse checklist

- ☐ Put `song.mp3` and `lyrics.txt` in a new folder; open Claude Code there.
- ☐ Run prompt B (timing). Read the confidence report.
- ☐ Run prompt C (sync test). Watch it with sound; note any wrong spots.
- ☐ Run prompt D (creative direction). Read the top-10 summary; push back on anything generic.
- ☐ Run prompt E (engine + hero proof) on your most important 5–40 seconds.
- ☐ Approve, then ask for sections by time range ("from 1:20 to 2:30").
- ☐ Keep `implementation-notes.md` up to date; it lists known weaknesses to fix next.

Time & cost reference (this project)

STAGE	MACHINE TIME	MAIN TOOLS
Timing	≈15 min (model downloads included)	Demucs, MMS aligner, Whisper
Sync test	≈25 s render	Pillow + libraqm, ffmpeg
Creative direction	Writing only	librosa (beat analysis)
Hero proof (5 s)	≈7 s render	Remotion, HarfBuzz glyph export
Section (70 s)	≈75 s render	Remotion (Chrome), 6 parallel tabs

Project files at a glance

lyrics.txt · song.mp3 (inputs)
word-timestamps.json · phrases.json (frozen timing)
beat-map.json · creative-direction.md · visual-screenplay.json (design)
sync-test.mp4 · hero-proof.mp4 · section-1m20-2m30.mp4 · hero-frames/ (outputs)
alignment/ (timing scripts) · video/ (Remotion engine) · implementation-notes.md

Appendix: the prompts, verbatim

These are the exact prompts used, in order. Copy them for your own project and swap in your file names and time ranges.

CREDIT & RIGHTS

The song belongs to its artist. This was made as a portfolio/fan piece. Credit the artist, and get permission before any commercial use. All illustrations were drawn by the process itself, and all fonts are open-licence (SIL OFL).

A • Start

```
hi check the folder
```

B • Word-level timing

```
I have provided two files in this project:
- song.mp3 – the original rap song
- lyrics.txt – the exact/canonical lyrics

Before creating any video, I need you to create highly accurate word-level timestamps for the lyrics.
Inspect the environment and install/use whatever local tools are required. Prefer WhisperX or another forced-
alignment approach suitable for fast rap vocals.
The lyrics.txt file is the source of truth for the WORDS.
The audio is the source of truth for the TIMING.
Do not simply transcribe the song and replace my lyrics. Align my supplied lyrics against the actual vocal
performance.
I need output in this format:
[
  {
    "word": "example",
    "start": 1.240,
    "end": 1.510
  }
]
Save the final result as:
word-timestamps.json
Also create:
phrases.json
where individual words are grouped into the actual lyrical lines/phrases while retaining their timing.
Because this is rap, pay special attention to:
- very fast delivery
- words spoken close together
- repeated words
- pauses
- ad-libs
- contractions
- elongated words
- differences between transcription and lyrics
First inspect the files and determine the best alignment approach available in this environment.
Then actually execute the transcription/alignment.
Do not start building the video yet.
At the end, verify that the timestamps cover the song correctly and report any portions where alignment
confidence appears poor.
```

C • Sync test video

```
Great. Do NOT modify or re-run the alignment yet.
Before we build the actual motion-graphics video, I want to visually verify the synchronization.
Create a very simple 9:16 verification video using:
- song.mp3
- word-timestamps.json
- phrases.json
VIDEO:
```

1080 × 1920
30 FPS
Black background
Original song.mp3 as audio
Display the CURRENT PHRASE around the center of the screen.
Within that phrase:
- inactive words = white/gray
- currently spoken word = visually highlighted
- change the highlight using the exact word-level start/end timestamps
- keep the entire phrase visible so I can understand context
Also display small debug information at the bottom:
current song time
line number
current word
word start → end
Example:
LINE 24
02:13.420
"तलब"
133.21 → 135.18
Do not add motion graphics, transitions, camera movements, effects, beat detection, or artistic styling yet.
This video's ONLY purpose is synchronization QA.
Render:
sync-test.mp4
Use the existing timestamps exactly as they currently exist. Do not silently alter them.
After rendering, tell me the exact output path.
Do not begin building the final lyric video after this. Stop after sync-test.mp4 is rendered.

D · Creative direction & screenplay

The synchronization test is approved.
The word-level timestamps are accurate enough. FREEZE the alignment data.
Do NOT modify:
- word-timestamps.json
- phrases.json
- the original audio
We are now moving to the FINAL MOTION DESIGN stage.
However, DO NOT immediately start coding/rendering the final video.
First I want you to behave like a senior motion designer + music video art director + creative developer and DESIGN THE ENTIRE VISUAL LANGUAGE OF THE SONG.
This is NOT supposed to be a normal lyric video.
My goal is to create a highly impressive portfolio piece that I can post publicly and tag music artists/producers. It should feel like someone spent serious time art-directing a motion-graphics music video.
Think:
high-end kinetic typography
+ lyric-driven visual metaphors
+ experimental music-video graphics
+ editorial typography
+ 2D/2.5D motion graphics
+ creative transitions
+ procedural graphics
+ occasional illustration/iconography
+ camera choreography
+ beat-aware editing
The most important principle:
THE MEANING OF THE LYRIC SHOULD DRIVE THE VISUAL.
Do NOT simply cycle through reusable animations such as zoom, slide, rotate, glitch, scale, fade, etc.
Those are animation primitives, NOT the creative concept.
For every line, ask:
"What is the rapper actually saying, and how could that meaning physically exist on screen?"
Example:
"जो चुभती है तुझ को, हाँ जैसे मेरा काम और ये मूँछ"
I do NOT want:
चुभती → zoom
काम → slide
मूँछ → shake
Instead, interpret it.
For "चुभती", typography could physically interact with something sharp – spikes, needles, thorns, pointed geometry, piercing typography, etc.

The word could itself become sharp.
Then instead of resetting the composition for the next word, continue developing the SAME VISUAL WORLD.
When "हूँ" arrives, an illustrated/stylized moustache could actually emerge, grow, curl around the typography, interact with the letters, or become part of the word.
Then perhaps the moustache sweeps across the camera and IT becomes the transition into the next scene.
That is the level of semantic thinking I want.

CONTINUOUS VISUAL STORYTELLING

Do not treat every word as a separate animation.
Think in:
WORD → PHRASE → SCENE → SEQUENCE → SONG
Some words should barely move.
Some phrases should share one composition.
Some important words deserve an entire visual concept.
Some sections can be typography-only.
Some sections can introduce graphical objects.
Some moments should deliberately become minimal so the next visual explosion feels stronger.
The viewer should constantly wonder:
"What is going to happen visually with the next bar?"

SEMANTIC VISUAL METAPHORS

Analyze ALL 82 lyric lines.
For important words/concepts, brainstorm visual representations.
Examples:
fire → letters burning / heat distortion / ember particles
falling → typography physically falling under gravity
breaking → letters fracture
cutting → text gets sliced
sharp/stinging → spikes/thorns/needles
heart → graphical heart integrated into composition
eyes → typography watched by illustrated eyes
money → currency / counting / receipts / numbers
phone/call → UI elements or incoming-call graphics
time → clocks / timelines / numbers / ticking composition
speed → typography stretches with motion trails
heavy → text physically becomes heavy and crushes something
high → composition/camera moves upward
low → typography falls below frame
alone → one tiny word surrounded by huge negative space
crowd → duplicated typography overwhelms frame
question → typography forms question marks / interrogation composition
answer → visual resolution of previous question scene
mirror → duplicated/reflected typography
love → avoid generic heart animation unless the lyric/context makes it appropriate; find something more original
These are examples ONLY.
Do not blindly use them.
Interpret the ACTUAL lyrics and cultural/contextual meaning.
The lyrics contain Hindi/Hinglish/Marathi/English, so preserve the original language and understand slang/context before designing a scene.

VISUAL ASSETS

You are NOT restricted to typography.
Where conceptually useful, use:

- vector illustrations
- SVG graphics
- geometric objects
- silhouettes
- icons
- hand-drawn elements
- textures
- particles
- procedural shapes
- grids
- diagrams
- UI graphics
- symbols
- abstract 3D-looking forms
- image cutouts where appropriate
- masks
- displacement/distortion
- shadows
- depth layers

BUT:

Do not turn this into generic stock footage.

Typography should still remain one of the heroes.

Graphics should INTERACT with typography.

For example, don't show a moustache icon beside "ਮੁੱਝ".

Make the moustache grow FROM the typography, sit on letters, push letters away, become a mask, or become the transition itself.

TRANSITIONS MUST HAVE MOTIVATION

Avoid:

scene → random wipe → next scene

scene → random zoom → next scene

scene → random glitch → next scene

Instead, objects from Scene A should frequently CREATE Scene B.

Examples:

a letter grows until it becomes the next background

a moustache sweeps across frame and becomes a curved mask

a falling word knocks the next lyric into frame

a crack travels through text and reveals the next scene

a circle becomes a moon/eye/record/spotlight in the following scene

a line extends from one word and becomes the baseline for the next composition

camera travels through a letter and emerges into another world

This should make the video feel continuously designed rather than assembled from templates.

MOTION DESIGN SYSTEM

Yes, create reusable animation primitives/components such as:

PunchWord

SlidePhrase

HugeImpact

WordCascade

MaskedReveal

TrackingExpand

CameraPush

GlitchHit

Typewriter

ElasticWord

TextPath

CharacterScatter

CharacterAssemble

PerspectiveText

DepthParallax

ImpactShake

MotionTrail

LiquidReveal

StrokeDraw

ShapeMorph

ParticleBurst

BUT THESE ARE ONLY TOOLS.

Build semantic compositions ON TOP of them.

I do not want Claude randomly selecting an animation component for each word.

Create higher-level SCENE components/concepts.

For example:

SharpTypographyScene

GravityScene

MoustacheMorphScene

QuestionAnswerScene

CrushScene

TunnelTypographyScene

InfiniteWordScene

MirrorScene

BurnScene

etc.

Create custom ones whenever the lyrics justify them.

BACKGROUNDS

The background should NOT remain a boring solid black background throughout the song.

Develop a strong visual identity.

Use dynamic backgrounds such as:

- gradients
- grain
- halftone
- paper/print texture
- animated geometry

- grids
- abstract shapes
- depth layers
- light sweeps
- radial compositions
- noise
- subtle particles
- poster-style graphics
- monochrome sections
- aggressive contrasting sections

Backgrounds should evolve with different sections of the track.

However, avoid visual noise that hurts readability.

TYPOGRAPHY

Typography must feel intentionally art-directed.

Create hierarchy using:

scale

weight

spacing

orientation

position

perspective

negative space

line breaks

character-level composition

Important words may occupy the entire frame.

Other words may become tiny.

Sometimes only ONE word should dominate.

Do not center every lyric.

Explore:

vertical typography

edge-aligned text

text wrapping around objects

perspective text

circular text

words behind foreground objects

words partially outside frame

characters used as graphical shapes

Hindi/Devanagari typography must render correctly.

Test all fonts before committing to them.

Do NOT break Devanagari shaping by animating individual Unicode codepoints incorrectly. If character-level effects are used, respect grapheme clusters/shaping or render words/phrases safely.

MUSIC & ENERGY

The timestamps tell you WHEN words happen.

The audio should tell you HOW the visuals behave.

Analyze the audio structure as well:

- intro
- verse
- hook
- transitions
- beat drops
- pauses
- energy changes
- melodic sections
- aggressive sections
- repeated hooks
- outro

Do not maintain maximum visual intensity for 3.8 minutes.

Create CONTRAST.

For example:

quiet section → restrained graphics

build-up → increasing visual complexity

hard bar/beat → strong visual payoff

hook → recognizable visual identity

verse → faster storytelling

pause → use negative space

drop → composition transformation

This contrast will make the crazy moments feel much crazier.

BEAT ANALYSIS

In addition to the existing word timestamps, analyze the original audio for useful beat/onset information.

Create separate beat/onset timing data if helpful.

Do NOT modify word timing.
Use:
WORD TIMESTAMPS → lyric synchronization
BEAT/ONSET DATA → secondary motion, cuts, impacts, background reactions and transitions
Do not make everything pulse on every beat.
FIRST TASK – DO NOT CODE FINAL VIDEO YET
Read ALL lyrics from lyrics.txt and ALL phrases from phrases.json.
Then create:
creative-direction.md
This should establish:
1. overall art direction
2. typography system
3. color strategy
4. texture/background system
5. animation principles
6. transition principles
7. intensity curve throughout the song
8. recurring visual motifs
Then create:
visual-screenplay.json
For ALL 82 lines.
Each line/scene should include approximately:
{
 "line": 0,
 "start": ...,
 "end": ...,
 "lyrics": "...",
 "meaning": "...",
 "heroWords": [...],
 "visualConcept": "...",
 "semanticMetaphor": "...",
 "typographyTreatment": "...",
 "background": "...",
 "graphics": [...],
 "motion": "...",
 "transitionIn": "...",
 "transitionOut": "...",
 "intensity": 1-10,
 "customAssetsNeeded": [...],
 "notes": "..."
}
But do NOT mechanically force every line to become a completely independent scene.
Explicitly identify which adjacent lines belong to the same visual sequence.
IMPORTANT CREATIVE REVIEW
After generating the screenplay, review it yourself.
Reject concepts that are:
- generic
- repetitive
- just zoom/slide/fade
- visually unrelated to lyrics
- obvious template effects
- impossible to read at rap speed
- culturally/contextually incorrect
- repetitive versions of earlier scenes
Check how often each visual idea is repeated.
If the same concept appears too often, redesign those scenes.
I would rather have 25-40 excellent interconnected visual sequences across the 82 lines than 82 unrelated template animations.
TECHNICAL FEASIBILITY
We will ultimately implement this in Remotion/React.
Therefore the concepts should be ambitious but programmatically achievable using combinations of:
React
Remotion
SVG
CSS transforms
Canvas where useful
procedural graphics
masks
filters
textures

```
generated/static assets
FFmpeg preprocessing if required
If a visual requires an external asset, explicitly list it.
Do NOT download random copyrighted imagery.
Prefer creating original SVG/procedural assets ourselves.
## FINAL GOAL
When someone watches this without knowing how it was made, I want them to think:
"This looks like a motion designer made a real music visualizer/music video."
NOT:
"This looks like AI animated some subtitles."
The typography, illustrations, backgrounds, transitions, lyric meaning and music should feel like ONE visual
system.
For now:
DO NOT render the final video.
Create only:
creative-direction.md
visual-screenplay.json
Then give me a concise summary of the strongest 10 visual moments you designed so I can review the creative
direction before implementation.
```

E · Engine + hero proof

```
Creative direction APPROVED.
The concept "The Case File of a Question," the three visual worlds, red-line motif, 37 sequences, beat map,
typography system and semantic concepts should now be treated as the creative source of truth.
Do not redesign the screenplay unless implementation reveals a genuine technical problem.
Use:
creative-direction.md
visual-screenplay.json
beat-map.json
word-timestamps.json
phrases.json
original MP3
assets/fonts/
The word timestamps, phrase timestamps and original audio remain FROZEN.
Now begin implementation in Remotion + React/TypeScript.
However:
DO NOT build/render the entire 228.9-second video immediately.
We need to establish the QUALITY BAR first.
# PHASE 1 – BUILD THE MOTION DESIGN ENGINE
Set up a production-quality Remotion project.
Create a clean architecture separating:
timing/
typography/
backgrounds/
graphics/
transitions/
effects/
sequences/
assets/
utils/
Build reusable LOW-LEVEL primitives for:
typography animation
SVG drawing
masks
clipping
camera movement
depth/parallax
grain
texture
paper
ink
red-line drawing
character/word reveals
shape morphing
motion trails
impact
controlled distortion
background transitions
```

But remember:
These primitives are implementation tools.
The visual-screenplay remains responsible for creative decisions.
Do NOT randomly assign effects to words.
CRITICAL: MOTION MUST FEEL DESIGNED
Avoid the common "AI-coded motion graphics" look:
linear translateX
basic scale(0→1)
generic spring on every word
constant opacity fades
everything centered
every word entering separately
random rotation
random glitch
hard scene resets
same easing everywhere
Motion needs:
anticipation
overshoot where appropriate
follow-through
stagger
weight
inertia
acceleration/deceleration
hierarchy
continuity
Different objects should FEEL physically different.
Paper ≠ ink ≠ metal ≠ thread ≠ skin ≠ typography.
For example:
A rubber stamp should have weight:
fast downward acceleration → impact → tiny compression → rebound → ink residue.
A thread should have elasticity.
Paper should have subtle dimensionality.
A heavy word should resist movement.
Ink should spread/draw rather than simply fade in.
BUILD A CAMERA SYSTEM
Do not make this feel like 37 PowerPoint slides.
Create a virtual 2D/2.5D camera abstraction.
Scenes can exist on a larger visual canvas.
The camera should sometimes:
push into typography
pull back
track a moving object
travel across a page
rotate with a physical object
pass through letterforms
reveal that one composition existed inside another
Transitions should often happen because the CAMERA or an existing object leads us into the next composition.
RED LINE
Treat the red line as a CHARACTER, not decoration.
Build it as a reusable animated SVG/path system capable of becoming:
underline
pen stroke
thread
suture
clock hand
crack
road
wound
question mark
Whenever possible, morph/continue the existing line rather than destroying it and spawning another.
Its continuity is one of the most important devices in the entire film.
TYPOGRAPHY
Use the fonts already tested in assets/fonts/.
Preserve proper Devanagari shaping.
Never animate individual Unicode codepoints in a way that breaks conjuncts/matras.
If character-level manipulation is required, use grapheme-safe segmentation or pre-render the correctly shaped unit.
Typography should have physical presence.
Use:

extreme scale differences
variable weight where supported
perspective
depth
clipping
negative space
edge compositions
foreground/background relationships
text on paths
text integrated into graphics
Do not default to center alignment.

BACKGROUNDS

Implement the three visual worlds from creative-direction.md:

PAPER
NIGHT
BLOOD-RED

But each is a SYSTEM, not a flat background color.

PAPER should have:

- subtle grain
- print imperfections
- ink behavior
- editorial structures
- physical depth

NIGHT should have:

- deep blacks
- controlled amber light
- falloff
- depth
- atmosphere
- selective visibility

BLOOD-RED should feel deliberately different and should be reserved for the ending as specified.

Do not overuse digital neon/glow aesthetics.

AUDIO REACTIVITY

Use:

- word-timestamps.json → primary lyric timing
- beat-map.json → secondary rhythmic motion

Do not animate everything on every beat.

Use downbeats/kicks selectively for:

- impact
- camera impulse
- background response
- scene transformation
- object collision
- major typography emphasis

Subtle secondary elements can respond to rhythm at lower amplitude.

ORIGINAL SVG ASSETS

Create the required line-art assets ourselves:

- hands
- eye
- crow
- coin
- arch
- fingers
- glass
- etc.

Keep them stylistically consistent with the Case File visual language.

Avoid generic emoji/icon appearance.

They should feel hand-drawn/editorial and capable of interacting with typography.

PHASE 2 – BUILD A HERO PROOF

Before implementing the full song, build ONLY a representative section.

Use approximately:

- 53s → 95s

This gives us:

- the moustache sequence
- melodic transition
- Hook I
- head/hand interaction
- circle transformation
- “दीवाना” payoff
- record transformation

rubber-stamp transition

This section should establish whether the final film is actually going to look professional.

Implement these sequences FULLY.

No placeholders.

No "we'll polish later" versions.

Include:

- final typography
- final backgrounds
- final SVG graphics
- final transitions
- final camera behavior
- final textures
- final timing
- final easing
- final audio synchronization

Render:

hero-proof.mp4

1080 × 1920

30 FPS

original audio

MOUSTACHE SEQUENCE – IMPORTANT

This is one of the signature moments.

Do not simply put a moustache SVG under the word.

For:

“जो चुभती है तुझ को, हाँ जैसे मेरा काम और ये मूँछ”

Build semantic progression.

“चुभती” should introduce sharpness/piercing.

Explore pointed geometry/spikes interacting physically with the typography.

The visual tension should travel through the phrase.

When “मूँछ” arrives:

use the actual Devanagari headline/shirorekha as the starting geometry.

Extend/morph it into the moustache.

The moustache should feel like it is born FROM the typography.

It should curl outward with confident physical motion.

Then use that SAME geometry to sweep across the camera and motivate the transition.

No moustache sticker/icon appearing beside the word.

HOOK SEAL SEQUENCE

This needs to feel like ONE continuous transformation:

melodic line

↓

circle

↓

head

↓

hand rests on head

↓

red fill

↓

दीवाना payoff

↓

record

↓

spin/flip

↓

rubber stamp

↓

impact starts Verse 2

Do not implement these as separate shots joined with cuts.

The viewer should be able to visually follow the object's identity throughout the transformation.

The transformation itself is the spectacle.

COMPOSITION SAFETY

This will primarily be viewed on Instagram Reels.

Keep critical text away from areas likely to be obscured by:

- username/caption at bottom
- right-side interaction buttons
- top UI

But intentionally allow decorative graphics and oversized typography to bleed beyond safe areas.

PERFORMANCE

Prefer SVG/CSS/Remotion-native techniques when sufficient.

Use Canvas only when it materially improves an effect.

```
Avoid unnecessarily expensive per-frame calculations.
Precompute paths/data where practical.
Make deterministic renders.
No Math.random() without deterministic seeding.
# QA
After implementation, inspect the hero proof for:
1. Lyric synchronization
2. broken Devanagari shaping
3. clipping
4. accidental overlaps
5. unreadable frames
6. animation discontinuities
7. weak transitions
8. repetitive movement
9. objects teleporting between states
10. motion that feels mechanically linear
11. visual elements obscured by Reel UI
12. frames where too much happens for the available lyric duration
Also inspect representative frames/screenshots rather than trusting code alone.
If a concept looks weak in the actual render, improve the IMPLEMENTATION while preserving the screenplay's
intent.
# IMPORTANT
Do not optimize for "technically completed."
Optimize for:
"Would a professional music artist actually want this visual style for their release?"
If a scene technically works but looks like a coding demo, it is not finished.
# DELIVERABLES FOR THIS PHASE
Create:
hero-proof.mp4
and
implementation-notes.md
The notes should briefly explain:
- architecture created
- components/effects built
- SVG assets created
- any deviations from screenplay and why
- known visual weaknesses
- performance/render considerations
Also export 8 representative PNG frames from the hero section so individual compositions can be reviewed
without pausing the video.
STOP after the hero proof.
Do NOT proceed with the remaining song until I approve the visual quality.
```

E2 • Scope-down (sent mid-way through E)

```
can we make a small section of the video quickly with all this & not take too much time
```

F • Render a section

```
looks good, but now i want from 1:20 minute to 2:30 minute can you make a video of that section now i approve
hero-proof.mp4
```